

Atlas

Business & 4-Week Implementation Plan

Autonomous Multi-Agent Research & Report Assistant

Digital Egypt Pioneers Initiative | AI & Data Science Track

At a glance	
Product	An agentic AI system that takes one research question and, with no further human input, plans, researches, writes, and verifies a source-cited report.
Why it is agentic	It decides its own steps, chooses its own tools, observes results, and changes course: a Critic can reject a draft and send work back to the Researcher or Writer.
Team / timeline	4-6 students, 28 days (4 weeks), feature freeze on Day 21.
Core stack	Python, LangGraph, Chroma (RAG), a web-search API, Pydantic, FastAPI, Streamlit, Docker, Langfuse or LangSmith, RAGAS, GitHub Actions.
Headline proof	An ablation study: single prompt vs. RAG-only vs. full Atlas, on the same 25 questions.
Deliverables	Deployed system, evaluation report, final report, presentation, live demo (mapped to the booklet's five milestones).

Booklet compliance note. The booklet allows ideas outside its list only if the initiative's management is informed and the idea follows the same format as the listed projects. Atlas is not one of the four listed projects, so inform the instructor / management early (Day 1) and keep the five-milestone structure (see Appendix A). The booklet also requires the task split to be reported to the instructor within two weeks of the announcement, and each member must explain their own part in the final presentation.

1. Executive Summary

Research is slow work: finding sources, reading them, cross-checking claims, and writing a coherent result. General chatbots answer quickly but often without sources, and their reasoning cannot be inspected. Atlas is a team of cooperating AI agents that does the whole research loop and shows its work. A **Planner** splits the question into sub-questions. A **Researcher** decides, step by step, whether to search the web, query a curated knowledge base, or run a calculation, and stops when it judges the evidence sufficient. A **Writer** drafts a report in which every claim cites an evidence item. A **Critic** checks each claim against its cited source and either approves the report or sends specific problems back to the right agent.

The business case is a transparent, verifiable, domain-tunable research assistant for students, analysts, and small teams. The realistic outcome of this graduation project is a strong, measurable, deployable system and a portfolio-grade case study, with a credible path to a niche pilot afterwards. The plan below fits the full build into four weeks by fixing the stack on Day 1, cutting scope to a defended core, and freezing features on Day 21.

2. Problem and Opportunity

- **Manual research does not scale.** A single well-sourced briefing can take hours of searching, reading, and reconciling sources.
- **Unsourced answers are hard to trust.** Plain chatbot answers rarely let the reader check where each claim came from.
- **Black-box research tools.** Many tools return a finished report but hide the plan, the sources rejected, and the checks performed.
- **Generic tools ignore your documents.** Teams often need answers grounded in their own reports, papers, or policies.

Opportunity: an assistant whose process is visible (a live trace of plan, searches, drafts, and critiques), whose claims are bound to evidence IDs, and whose knowledge base the user controls.

3. The Product

Input: one natural-language research question. **Output:** a multi-section report with inline citations, a source list, a confidence note, and a downloadable trace of how it was produced.

User flow

- The user enters a question in the web UI (optionally choosing the knowledge base and a depth setting).
- The UI streams the agents' steps live: sub-questions, tool calls, evidence found, draft versions, critic verdicts.
- The system returns the approved report, or a graceful "insufficient evidence" report that states what could not be verified.
- The user exports the report (Markdown/PDF) and can open the trace and cost summary for auditing.

Core features (MVP)

- Structured planning (sub-questions with priorities).
- Tool-using researcher with retrieval-first, web-second behavior.

- Evidence-bound writing: every claim carries a source ID.
- Verification loop with conditional routing and a retry cap.
- Live agent trace, cost and latency display, tracing dashboard, and an evaluation harness.

4. Why Atlas Is Genuinely Agentic

A fixed script that calls a model in sequence is a pipeline, not an agent. Atlas is designed so that control flow depends on what the agents observe. The table maps each agentic property to a concrete design decision.

Agentic property	How Atlas implements it	Built in
Goal-directed autonomy	Given only a question, it runs to a finished, verified report with no human steps in between.	Week 2
Planning and decomposition	Planner emits a structured list of sub-questions; the number and content vary per question.	Week 2
Tool use with real choice	Researcher chooses between retrieve, web_search, and python_calc at each step, based on what it has already found.	Week 1
Observe-and-adapt loop (ReAct)	Reason, act, observe, repeat; the agent decides when evidence is sufficient instead of running a fixed number of steps.	Week 1
Reflection with routing	Critic returns a structured verdict; the graph routes to Researcher (missing evidence) or Writer (clarity) or finishes.	Week 2
Bounded autonomy	Retry cap, token/cost budget, timeouts, and an "insufficient evidence" exit prevent runaway loops.	Week 2
Memory	Short-term: graph state shared by all agents. Long-term (stretch): past evidence and reports in SQLite.	Week 1-2
Transparency	Every step is traced and shown live, so the autonomy is inspectable, not hidden.	Week 3

Anti-patterns to avoid

- A hard-coded chain (Plan then Search then Write) with no branching. If every run takes the same path, it is not agentic.
- A Critic that always approves. Seed test cases that must be rejected and check that they are.
- Passing free text between agents. Use typed objects (evidence with source IDs, verdicts with issue lists) so checks are verifiable.
- Unbounded loops. Every loop needs a cap and a defined exit.

Proof of agency (include in the final report)

- **Path variability:** across the 25 test questions, report the spread of tool sequences and loop iterations. Different questions should take different paths.
- **Critic effect:** the percentage of reports whose final score is higher than their first draft, and the number of runs where the Critic triggered a revision.
- **Ablation:** single prompt vs. RAG-only vs. full Atlas on faithfulness, coverage, and citation precision.

5. Target Customers and Use Cases

Segment	Need	Example use	Notes
Students and academic researchers	Fast, cited literature overviews	Background review of a topic, with sources to verify	Best fit for the graduation demo; easy to recruit pilot users
Analysts and consultants at small firms	Quick market or competitor scans	Two-page briefing on a sector	Values verifiability and cost control
Startup founders	Cheap, fast due diligence	Competitor and regulation scan	Price sensitive
Teams with internal documents	Answers grounded in their own files	Policy or report Q&A with citations	Strongest differentiator (own knowledge base); needs privacy care

These segments are hypotheses. Validate them in Week 3 with 5-10 pilot users (classmates, instructors, a few professionals) using a short feedback form: usefulness, trust in citations, time saved.

6. Competitive Landscape and Differentiation

Alternative	Strength	Gap Atlas targets
Deep-research modes in general AI assistants (e.g., ChatGPT, Gemini)	Polished, fast, huge model access	Limited visibility into steps; little control over the knowledge base; per-user cost control not exposed
AI search engines with citations (e.g., Perplexity)	Fast cited answers	Shallow multi-step planning and no explicit verification pass in the user's view
Academic literature tools (e.g., Elicit, Consensus)	Curated scholarly data	Narrow domain; not general report generation
DIY agent frameworks	Flexible	Require engineering effort; Atlas is a working, evaluated reference system

Competitor features change quickly. Re-check current capabilities before presenting this table.

Differentiation

- **Verifiability:** claim-to-source binding and an explicit verification pass.
- **Transparency:** live trace of plan, tool calls, and critiques.
- **Controllable knowledge base:** retrieval-first on the user's own documents.
- **Cost awareness:** per-report cost and latency shown, with budget caps.
- **Measured quality:** a published evaluation with baselines, not just a demo.

Honest positioning: Atlas will not out-scale large platforms that bundle research agents into subscriptions. Its defensible value is trust, transparency, and domain focus.

7. Business Model and Unit Economics (Illustrative)

For a graduation project the point of this section is to show cost awareness and a plausible model, not to claim validated pricing. Replace the assumptions below with the cost per report you measure in Week 3.

Plan (hypothetical)	Price / month	Included reports	Cost at \$0.25 per report	Gross margin
Free	\$0	3	\$0.75	n/a (acquisition cost)
Pro (individual)	\$19	40	\$10.00	about 47%
Team (5 seats)	\$59	150	\$37.50	about 36%

Cost per report = sum over all model calls of (tokens x price) + search API calls. Expect roughly 12-20 model calls per report. The \$0.25 figure is an assumption to be measured, and it moves a lot with model choice.

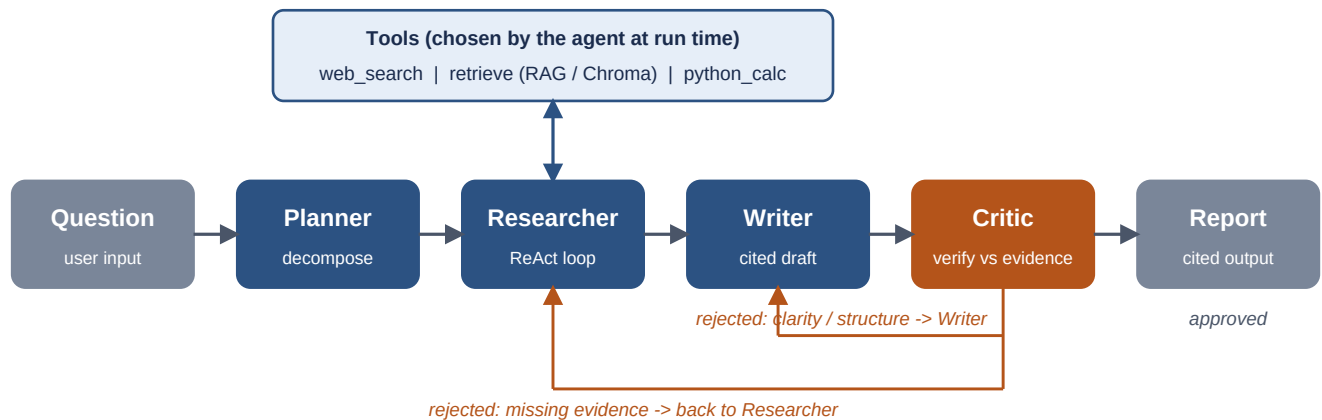
Levers to keep cost down

- Use a stronger model for Planner, Critic, and Writer, and a cheaper, faster model for routine Researcher steps.
- Retrieval first, web search second; cache search results and embeddings.
- Cap retries and total tokens per report; stop early with an "insufficient evidence" report.
- Send the Critic only the claims and their cited evidence, not the whole corpus.

8. Go-to-Market (Lean)

Phase	Timing	Actions	Success signal
Pilot	Weeks 3-4	Invite 5-10 pilot users; collect feedback form and trace usage	Most pilot users rate citations trustworthy and report time saved
Showcase	Weeks 5-8 (after graduation deadline)	Open-source repo, demo video, short write-up of the ablation results	Stars, forks, recruiter and peer feedback
Niche pilot	Later	Pick one vertical (e.g., a single research domain) and load a curated knowledge base	A pilot organization uses it on real questions

9. Technical Architecture



Shared state (LangGraph)

sub_questions | evidence[] (each item carries a source ID) | draft | verdict | retry_count | budget

Guardrails

retry cap | token / cost budget | timeouts | retrieved text treated as untrusted data | every step traced

Component	Responsibility	Technology
Planner agent	Turn the question into 3-6 structured sub-questions with priorities	LLM with Pydantic structured output
Researcher agent	ReAct loop per sub-question; choose tools; stop when evidence is sufficient; store evidence with source IDs	LangGraph node plus tools
Tools	retrieve(query, k), web_search(query), python_calc(expr)	Chroma, Tavily or SerpAPI, restricted evaluator
Writer agent	Produce the report; each claim cites an evidence ID	LLM
Critic agent	Check each claim against its cited evidence; score grounding, coverage, coherence; return a routed verdict	LLM judge plus programmatic citation check
Orchestrator	State machine with conditional edges, retry and budget caps	LangGraph
Memory	Short-term graph state; long-term store of past runs (stretch)	SQLite
API and UI	POST /research and a streamed step feed; chat-style front end with live trace	FastAPI, Streamlit
Observability	Traces, latency, token and cost per run	Langfuse or LangSmith
Evaluation	Question set, baselines, metrics, human spot-check	RAGAS plus custom scripts
DevOps	Containers, CI smoke evaluation, deployment	Docker, GitHub Actions, free-tier host

Typed contracts between agents

- **Evidence item:** id, sub_question_id, text snippet, source (URL or document ID), retrieval method, timestamp.

- **Critic verdict:** approved (true/false); issues[] each with type (unsupported_claim, missing_subquestion, clarity), location, detail, route_to (researcher or writer); scores for grounding, coverage, coherence.
- **Run state:** question, sub_questions, evidence[], draft, verdict, retry_count, tokens_used, cost_estimate.

Recommended stack decisions (lock on Day 1)

- **Orchestration:** LangGraph, because conditional edges and shared state map directly onto the Critic-routing design.
- **Models:** pick one strong model and one cheaper model from your chosen provider; keep model names in a config file so they can be swapped.
- **Knowledge base domain:** choose a narrow domain where the team can judge correctness, for example recent open-access papers on LLM agents, open government reports, or a single industry's public filings. Cap the corpus at about 100-300 documents.

10. Scope: What Ships in Four Weeks

Priority	Items
Must have	Planner; Researcher with retrieval and web search; Writer with evidence-ID citations; Critic with conditional routing and retry cap; FastAPI plus Streamlit UI with live trace; tracing; evaluation harness with three-way ablation; Docker deployment; final report, slides, live demo.
Should have	Search and embedding cache; cost dashboard; prompt versioning; CI smoke evaluation; Markdown/PDF export; basic prompt-injection defenses.
Could have	Long-term memory across sessions; parallel researchers per sub-question; human approval of the plan before research starts; Arabic-language support.
Will not do now	Model fine-tuning; user accounts and billing; custom vector infrastructure; mobile app.

Compared with the earlier nine-week outline, this plan compresses Milestones 1-2 into one week and treats long-term memory as optional, so the agentic core (planning, tool choice, critique-and-route) is never at risk.

11. Four-Week Execution Plan

Assumptions: 5 team members at about 15-20 hours per week each; LLM API credits available; free-tier hosting. Work in small pull requests, hold a 15-minute daily stand-up, and demo working software at the end of every week.

Week 1 (Days 1-7): Foundation and single-agent core (Booklet Milestones 1-2)

Days	Work	Lead
1	Kickoff: pick the knowledge-base domain; lock the stack; create repo, environment, and secrets handling; assign roles; report the task split and the custom idea to the instructor / management; write the Definition of Done.	All
2-3	Collect and chunk 100-300 documents, embed, index in Chroma, test retrieval. In parallel: tool wrappers with Pydantic schemas; FastAPI and Streamlit skeletons; Dockerfile; draft the 25-question evaluation set and rubric.	Retrieval, Agent, Backend, Eval, UI
4-5	ReAct Researcher working end to end: retrieval-first, web search fallback, evidence stored with source IDs; basic answer with citations.	Agent, Retrieval
6	Integrate UI, API, and agent; run 10 questions; log failures.	All
7	Buffer and Gate 1 review.	All

Gate 1: a single agent answers questions with cited sources through the UI; retrieval returns relevant passages; the 25-question set and rubric are written; the deployment skeleton builds in Docker.

Week 2 (Days 8-14): Multi-agent orchestration and reflection (Milestone 3)

Days	Work	Lead
8	Define the LangGraph state schema; implement the Planner with structured output.	Agent
9-10	Writer with evidence-ID citations; Critic with rubric and structured verdict; programmatic check that every cited ID exists.	Agent, Eval
11-12	Conditional routing (approve, back to Researcher, back to Writer); retry cap; token and time budgets; graceful "insufficient evidence" exit; tool-failure handling.	Agent, Backend
13	Stream the agent trace into the UI; seed 3-5 test questions the Critic must reject.	UI, Eval
14	Buffer and Gate 2 review.	All

Gate 2: the full loop completes on at least 80% of the 25 questions; at least three runs show a Critic-triggered revision that improved the report; seeded bad drafts are rejected.

Week 3 (Days 15-21): Evaluation, observability, deployment, hardening (Milestone 4)

Days	Work	Lead
15-16	Tracing (Langfuse or LangSmith) with cost and latency; automated evaluation harness (RAGAS faithfulness and relevance, citation precision, coverage); prompt versioning.	Backend, Eval
17-18	Run the three-way ablation (single prompt, RAG-only, full Atlas); calibrate Critic thresholds; human spot-check of 10 reports by two raters; pilot users start testing.	Eval, All
19	Deploy the container to a free-tier host; CI runs a smoke evaluation on each push; enable caching.	Backend
20	Hardening: no results, contradictory sources, off-topic questions, prompt injection in web pages, rate limits.	Agent, Retrieval
21	Gate 3 and feature freeze. After this day: bug fixes only.	All

Gate 3 (freeze): deployed system reachable by URL; evaluation results recorded for all three configurations; measured cost and latency per report known; tracing dashboard live.

Week 4 (Days 22-28): Report, presentation, demo (Milestone 5) and buffer

Days	Work	Lead
22-23	Write the final report in parallel by section owner; produce ablation tables and charts; document limitations and failure cases.	All
24	Slides with architecture diagram and headline metrics; demo script; record a backup demo video; prepare a cached fallback run.	UI/Docs, Agent
25	Dress rehearsal 1 on unseen questions. Fix only defects found.	All
26	Dress rehearsal 2 plus Q&A drill: each member explains their own module and decisions.	All
27	Package: README, release tag, final report and slides exported, submission checks.	Docs, Backend
28	Buffer day / presentation day.	All

12. Team and Responsibilities

Role	Owns	Key deliverables
Agent and Orchestration Lead	Planner, graph, routing, guardrails, prompts	Working multi-agent graph; retry and budget logic
Retrieval and Tools Engineer	Corpus, chunking, embeddings, Chroma, search and calculator tools, caching	Indexed knowledge base; tested tool wrappers
Evaluation and Quality Engineer	Question set, rubric, RAGAS and custom metrics, ablation, Critic calibration, human spot-check	Evaluation report and ablation table
Backend and MLOps Engineer	FastAPI, Docker, CI, tracing, deployment, cost logging	Deployed service; CI pipeline; tracing dashboard
Frontend and Documentation Lead	Streamlit UI with live trace, final report, slides, demo script, pilot feedback form	UI; report; presentation; backup video

With 4 members, combine the last two roles into a single Backend and UI role. With 6, split documentation and project management into its own role. Whatever the split, record it in writing: the booklet requires it to be reported to the instructor and presented at the end.

13. Budget and Resources (Estimates)

Item	Estimate	Notes
LLM API usage	About \$100-150 total	Assumes about \$0.25 per full report, several evaluation sweeps, and development testing. Scales strongly with model choice; use a cheaper model during development.
Web search API	Free tier likely enough	Verify current limits; cache results to stay within them.
Hosting	Free tier	Container host or Spaces-style platform; verify current limits and sleep behavior.
Embeddings and compute	No GPU needed	Embeddings can run on CPU or a free notebook environment; generation is via API.
Tooling	Free	GitHub, Langfuse (cloud free tier or self-hosted) or LangSmith, RAGAS, LangGraph are open source or have free tiers.

Control spend with per-key monthly limits, a shared team budget, response caching, and a hard per-run token budget in the graph.

14. Success Metrics and Evaluation Plan

Test set (25 questions): 15 answerable from the knowledge base, 5 that need web search, and 5 adversarial (unanswerable, contradictory sources, off-topic, or containing an injected instruction).

Configurations compared: (A) single-prompt LLM, (B) retrieval plus one writer, no Critic, (C) full Atlas.

Metric	Target	How measured
Faithfulness / grounding	At least 0.80 (RAGAS-style)	Automated, plus human spot-check of 10 reports
Citation precision	At least 90% of cited claims supported	Programmatic ID check plus human review
Coverage	At least 80% of sub-questions addressed	Rubric scoring
Completion rate	At least 90% of runs end in a report or a graceful "insufficient evidence"	Run logs
Critic value	Final score above first draft in at least 60% of revised runs	Score comparison across iterations
Uplift vs. baseline	Full Atlas beats configurations A and B on grounding and coverage	Ablation table
Latency	Median under 3 minutes per report	Tracing
Cost per report	Measured; target under \$0.50	Tracing
Pilot feedback	Most pilot users rate citations trustworthy	Feedback form

Targets are goals to aim for, not guarantees. Report actual results honestly, including where Atlas does not beat a baseline.

15. Risk Register

Risk	Likelihood	Impact	Mitigation
Critic rubber-stamps weak drafts or loops forever	High	High	Structured verdicts; seeded must-reject tests; retry cap; calibrate thresholds against human ratings in Week 3
Cost or rate limits break testing and demos	Medium	High	Caching; cheaper dev model; per-run budgets; cached fallback run for the demo
Noisy or contradictory web results	High	Medium	Retrieval first; source allow/deny list; require agreement or flag conflicts; "insufficient evidence" exit
Prompt injection inside retrieved pages	Medium	High	Treat retrieved text as data, never as instructions; strip or flag instruction-like text; no privileged tools reachable from content
Scope creep (memory, extra agents, UI polish)	High	High	MoSCoW list; freeze on Day 21; new ideas go to a "later" list
Integration problems discovered late	Medium	High	End-to-end thin slice by Day 6; weekly demos; CI from Week 1
Team member unavailable	Medium	Medium	Two people understand each module; pair on the graph and the evaluation harness
Live demo failure	Medium	High	Pre-tested questions, cached run, backup recording, offline-capable fallback
Weak evaluation credibility	Medium	Medium	Fixed test set written before tuning; two human raters; report baselines and failures

16. Legal, Ethics, and Responsible AI

- **Attribution:** every claim links to a source; show the source list and retrieval date.
- **Content rights:** prefer open-access or properly licensed documents for the knowledge base; use search APIs rather than scraping, and respect site terms.
- **Privacy:** do not load personal or confidential data into the shared knowledge base or the logs; protect API keys and never commit them.
- **Honesty about limits:** label output as AI-generated, show a confidence or "unverified" flag, and never present unsupported claims as verified.
- **Safety of tools:** the calculator tool is sandboxed and cannot reach files or the network; web content cannot change the agent's instructions.

17. Post-Graduation Roadmap

- **Months 1-2:** open-source release, write-up, feedback from pilot users, fix top failure modes.
- **Months 3-4:** long-term memory, parallel research on sub-questions, human-in-the-loop plan approval, Arabic-language support.
- **Months 5-6:** one vertical pilot with a curated knowledge base and organization-level access controls.

18. Definition of Done and Demo Script

Definition of Done

- Deployed URL works; a new user can submit a question and receive a cited report.
- Live trace shows plan, tool calls, drafts, and at least one Critic verdict.
- Evaluation results exist for all three configurations on all 25 questions, with methodology written up.
- Cost and latency per report are measured and reported.
- Final report, slides, README, and backup demo video are complete; every member can explain their module.

Demo script (about 10 minutes)

Step	Content	Time
1	Problem and target user	1 min
2	Architecture walkthrough using the diagram	1.5 min
3	Live run on a new question suggested by the panel; narrate the trace	3 min
4	Pre-tested question where the Critic rejects a draft and the system repairs it	1.5 min
5	Ablation results and cost/latency numbers	1.5 min
6	Team roles, limitations, next steps	1 min
7	Q&A	remaining

Appendix A: Mapping to the Booklet's Five Milestones

Booklet milestone	Atlas equivalent	Days	Key deliverables
1. Data collection and preprocessing	Knowledge-base and environment setup (documents collected, cleaned, chunked, embedded)	1-3	Indexed knowledge base; documented preprocessing steps
2. Model development and training	Core agent development (ReAct researcher with tools; no model training required)	4-7	Working single-agent system with citations; test question set
3. Advanced techniques and pipeline integration	Multi-agent orchestration, reflection loop, guardrails	8-14	Multi-agent graph; example transcripts of revision cycles
4. MLOps and model management	Tracing, evaluation harness, prompt versioning, Docker and CI, deployment	15-21	Deployed system; tracking dashboard; evaluation report
5. Final report, presentation, and demo	Report, slides, live demo, rehearsals	22-28	Final report; presentation; live demo

Where the booklet's listed projects call for GAN training or fine-tuning, Atlas replaces model training with agent design, tool use, and evaluation, which is the equivalent technical depth for an agentic system. Mention this explicitly when informing the instructor and management about the custom idea.

Appendix B: Day-1 Checklist

- Inform the instructor / initiative management that the team is proposing a custom agentic AI idea in the booklet's five-milestone format.

- Choose the knowledge-base domain and confirm the team can judge answer quality in it.
- Choose the model provider and set per-key spending limits; store keys in environment variables, never in the repo.
- Create the repository, branch rules, and a CI skeleton; agree on code style and pull-request review.
- Assign roles and write them down; schedule the daily stand-up and the end-of-week demo.
- Write the Definition of Done and the MoSCoW scope list; pin them at the top of the README.